Chapter 6 Solutions

CH06-Q01

$$C = 10000(4)(500)(512) = 10{,}240{,}000{,}000 \text{ bytes} = 10.24 \text{ GB} \approx 9.5367 \text{ GiB}.$$

Rotation takes $60/12000 = .005$ s, five ms. Mean rotational delay is 2.5 ms. One sector occupies $5/500 = .010$ ms in the simplified uniform-sector model.

A random read takes $4.5 + 2.5 + .010 + .10 = 7.110$ ms. A continuous 100-sector request takes $4.5 + 2.5 + 100(.010) + .10 = 8.100$ ms. Seek, initial rotation, and setup are amortized. The stated same-track and continuous-request assumptions avoid extra positioning, gaps, and command overhead.

CH06-Q02

$$A = \frac{100000}{100004} \approx .9999600016.$$

Expected downtime under the long-run fraction is $8760(1 - A) = .350386$ hours, approximately 21.02 minutes. With MTTR .5 hour,

$$A' = \frac{100000}{100000.5} \approx .9999950000,$$

downtime is approximately 2.63 minutes/year, a reduction of 18.39 minutes.

This does not bound the annual outage count. MTTF does not specify interval variance, dependence, maintenance policy, or repair-time distribution. A count prediction needs an explicit failure process and incident evidence.

CH06-Q03

$WA = 384/120 = 3.2$. The simplified physical write budget gives

$$E_{host} = \frac{400(3000)}{3.2} = 375000 \text{ GB} = 375 \text{ TB}.$$

With unchanged 400-GB physical NAND and WA 4, endurance is $400(3000)/4 = 300000$ GB, 300 TB, 20% lower. Do not replace the physical numerator by 320 GB and subtract spare twice.

More spare usually helps garbage collection. This comparison also changes the working set and supplies an observed WA, so it does not prove that spare caused the increase. Near-full random updates can require copying many valid pages before erasure. Benchmark after a specified warm-up, holding free-space ratio, mix, request size, and queue depth explicit. Record host/NAND bytes, collection activity, and tail latency over the same interval. The arithmetic is not a warranty or lifetime prediction.

Chapter 6 Solutions (continued)

CH06-Q04

Polling costs $20000(100) = 2 \times 10^6$ cycles/s. Interrupts cost $20000(1500) = 3 \times 10^7$. At 3 GHz these occupy approximately 0.0667% and 1% of one CPU.

Under exactly one successful poll per operation, polling has lower stated overhead regardless of the number of other runnable threads. Additional threads alone do not create unspecified spin cycles. Repeated polling would introduce wait duration, check frequency, energy, and scheduling opportunity costs that this question does not quantify.

A hybrid can spin briefly for likely short completions, then sleep and await an interrupt, with optional completion coalescing. Choose the transition from completion-time distributions, queue depth, CPU availability, and latency requirements.

CH06-Q05

$64\ \mathrm{MiB} = 67{,}108{,}864$ bytes. DMA bandwidth is

$$67{,}108{,}864/.020 = 3.3554432 \times 10^9\ \mathrm{B/s}.$$

Setup takes $500/(3 \times 10^9) = 1.6667 \times 10^{-7}$ s, approximately .167 microsecond.

Each 16-byte programmed transaction has four four-byte moves, costing $4(4) + 10 = 26$ cycles. There are $67{,}108{,}864/16 = 4{,}194{,}304$ transactions. Total cycles are

$$4{,}194{,}304(26) = 109{,}051{,}904,$$

or approximately 36.35 ms CPU time at 3 GHz. Device waits and OS overhead are additional.

DMA saves core instructions but still uses memory/interconnect bandwidth. Correctness needs legal DMA mappings/IOMMU configuration, buffer lifetime, coherent visibility or OS-directed maintenance, descriptor ordering, completion observation, and error/timeout handling.

CH06-Q06

Device capacity is $6 \times 80000 = 480000$ IOPS. The common request path is bounded by $\min(480000, 120000, 300000, 150000) = 120000$ IOPS, HBA-limited. Two extra SSDs raise device capacity to 640000 but leave the 120000 HBA bound unchanged.

Each proposal below independently starts from the original configuration.

Chapter 6 Solutions (continued)

Explicit upgrade	Devices	HBA	Memory	CPU/driver	New bound
HBA to 200000, others fixed	480000	200000	300000	150000	150000, CPU
Two 120000 HBAs, three SSDs each; two independent 150000 CPU paths, balanced work, no extra shared limits	480000	240000	300000	300000	240000, HBAs
HBA and CPU each to 500000	480000	500000	300000	500000	300000, memory

These are assumed capacities used to infer the next bottleneck, not measured hardware promises. Driver-only optimization or batching still leaves the original HBA at 120000. Larger requests change bytes/s and workload; they do not establish higher same-workload IOPS.

CH06-Q07

Nominal capacities are 64 TB for RAID 0, 56 for RAID 5, 48 for RAID 6, and 32 for four mirrored pairs in RAID 10.

RAID 0 tolerates no member loss. RAID 5 recovers one member. RAID 6 recovers two erasures within its check-symbol model. RAID 10 can survive several losses if no mirror pair loses both members; any double loss in one pair destroys that stripe's redundancy.

RAID 5 read-modify-write reads old data and parity, then writes new data and parity: two reads plus two writes. The analogous RAID 6 model reads old data/P/Q and writes new data/P/Q: three reads plus three writes. Caching and full-stripe strategies can change real traffic.

CH06-Q08

Reconstructing nine TB on the failed member requires $11(9) = 99$ TB of surviving reads. That is traffic, not replacement capacity. The replacement receives only nine TB.

$$traffic = 99 + 9 = 108\ \mathrm{TB}, \qquad T \geq \frac{108 \times 10^{12}}{10^{9}} = 108000\ \mathrm{s} = 30\ \mathrm{h}.$$

Whole-disk rebuilding reads 110 TB and writes ten TB, totaling 120 TB, so its corresponding lower bound is 120000 s, 33.33 hours.

Chapter 6 Solutions (continued)

If "1 GB/s rebuild rate" instead means replacement output alone, nine TB takes 9000 s, 2.5 hours, and the surviving path must support eleven GB/s concurrently. That is a different denominator from the question's shared aggregate path.

RAID 6 read sets depend on the failed members, P/Q reconstruction algorithm, controller policy, and valid-extent tracking. Some single-loss paths can skip one check block; others read more survivors. Capacity ratios alone do not establish traffic or time.

Exposure depends on rebuild duration, surviving-device hazards, latent errors, and correlated power, cooling, controller, or batch failures. A mean lifetime does not specify those distributions or dependencies.

CH06-Q09

Payload contains $64(2^{20})(8) = 536{,}870{,}912$ bits. Effective rate is eight Gbit/s, one decimal GB/s, approximately .9313 GiB/s.

Pure transfer takes $67{,}108{,}864/10^9 = .067108864$ s. Sixteen batches have fifteen intervening gaps, adding three ms. Total is 70.108864 ms. Adding initial setup gives 70.308864 ms.

More streams can hide waits or fill an underutilized path, but cannot exceed shared link capacity. Queueing, CPU/protocol overhead, and storage limits can increase latency and its tail. Dividing the single-stream time by 100 would invent bandwidth.

CH06-Q10

$$\mu = 1/0.004 = 250/s, \quad \rho = 200/250 = 0.8,$$
$$W = 1/(250-200) = 20\ \text{ms}, \quad W_q = 20 - 4 = 16\ \text{ms}, \quad L = 200(0.020) = 4.$$

Payload is $200 \times 4096 = 819200$ B/s. Do not multiply throughput again by queued requests. The target requires

$$1/(250-\lambda) \le 0.010 \Rightarrow 250 - \lambda \ge 100 \Rightarrow \lambda \le 150/s.$$

At equality, rho 0.6, W 10 ms, L1.5. An average need not be integral, and this does not bound the 99 th percentile. Raising lambda 200→240 with unchanged 4 ms service raises W 20→100 ms and L 4→24. Little's law needs appropriate stable finite means, not Poisson arrivals; the M/M/1 response formula needs the stronger assumptions.

Chapter 6 Solutions (continued)

CH06-Q11

P=1010 XOR 1100 XOR 0110 XOR 1111=1111. Recomputed P'=1010 XOR 0101 XOR 0110 XOR 1111=0110. Shortcut 1111 XOR 1100 XOR 0101 also gives 0110, because the extra old value cancels the old contribution.

RMW reads k old data plus parity and writes k new data plus parity, total 2 k+2. Reconstruction reads 4−kunchanged blocks and writes k+1, total 5.

k	RMW reads/writes	RMW bytes	Reconstruct reads/writes	Reconstruct bytes
1	2/2	16384	3/2	20480
3	4/4	32768	1/4	20480

These are aggregate work, not parallel elapsed time. Placement, caches, and stripe locks affect actual timing. Durable new D1 makes XOR(data)=0110 while old P=1111, breaking parity consistency. Losing D0 then reconstructing from old parity gives incorrect data. Duplicate parity may leave both copies old; protected redo information or an explicitly ordered atomic protocol is required.

CH06-Q12

$P = 5 \oplus 7 \oplus 9 = 11$. In the stated field, $2 \cdot 7 = 14$. For $2 \cdot 9$, left shift binary `1001` to `10010`, keep `0010`, and XOR `0011` to get 1. Hence $4 \cdot 9 = 2$ and $Q = 5 \oplus 14 \oplus 2 = 9$.

Known $D_1 = 7$ gives $X = P \oplus 7 = 12$ and $Y = Q \oplus (2 \cdot 7) = 9 \oplus 14 = 7$. The equations are

$$D_0 + D_2 = 12, \qquad D_0 + 4D_2 = 7,$$

so $5D_2 = 7 \oplus 12 = 11$. To verify $5^{-1} = 11$, compute $2 \cdot 11 = 5$ and $4 \cdot 11 = 10$, so $5 \cdot 11 = 10 \oplus 11 = 1$. Thus $D_2 = 11 \cdot 11 = 9$ and $D_0 = 12 \oplus 9 = 5$. Substitution returns $P = 11, Q = 9$.

Erasure positions are known; only values are missing. Silent corruption may also hide the erroneous position. Two-erasure recovery does not imply arbitrary two-error location/correction.

CH06-Q13

Reads total $7 \times 6 = 42$ TB, writes total 6 TB, and combined traffic is 48 TB:

$$T = 48 \times 10^{12} / (2 \times 10^{9}) = 24000 \text{ s} = 20/3 \text{ h}.$$

All seven survivors last through T with probability exp(−7 lambda T), so

Chapter 6 Solutions (continued)

$$P = 1 - e^{-7(10^{-5})(20/3)} = 0.000466558 = 0.04666\%.$$

Longer rebuilds increase this probability under fixed hazards. If 2 GB/s is output, 6 TB takes 3000 s=5/6 h. Survivor reads need 14 GB/s and total traffic 16 GB/s. The calculation conditions on an existing first failure and one repair; annual risk also needs initial-failure frequency, repair cycles, correlation, and other errors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 6 Solutions (continued)

Verification code appendix: storage_math.py

```
"""Re-runnable storage and I/O calculations used in Chapter 6."""

from dataclasses import dataclass


def average_rotation_ms(rpm: float) -> float:
    return (60.0 / rpm) * 1000.0 / 2.0


def availability(mttf_hours: float, mttr_hours: float) -> float:
    return mttf_hours / (mttf_hours + mttr_hours)


def raid_capacity_tb(level: int, disks: int, disk_tb: float) -> float:
    if level == 0:
        data_disks = disks
    elif level == 1:
        data_disks = disks / 2
    elif level == 5:
        data_disks = disks - 1
    elif level == 6:
        data_disks = disks - 2
    else:
        raise ValueError(f"unsupported level: {level}")
    return data_disks * disk_tb


@dataclass(frozen=True)
class IOPath:
    device_iops: int
    controller_iops: int
    link_iops: int
    driver_iops: int

    def bottleneck(self) -> tuple[str, int]:
        limits = {
            "device": self.device_iops,
            "controller": self.controller_iops,
            "link": self.link_iops,
            "driver": self.driver_iops,
        }
        name = min(limits, key=limits.get)
        return name, limits[name]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 6 Solutions (continued)

```
def main() -> None:
    rotation = average_rotation_ms(12000)
    assert abs(rotation - 2.5) < 1e-12
    one_sector_transfer_ms = (60.0 / 12000) * 1000.0 / 500
    assert abs(one_sector_transfer_ms - 0.01) < 1e-12
    access_ms = 4.5 + rotation + one_sector_transfer_ms + 0.10
    assert abs(access_ms - 7.11) < 1e-12

    avail = availability(100000, 4)
    downtime_minutes = (1 - avail) * 8760 * 60
    assert abs(downtime_minutes - 21.0223) < 0.001

    capacities = {
        "RAID0": raid_capacity_tb(0, 8, 8),
        "RAID5": raid_capacity_tb(5, 8, 8),
        "RAID6": raid_capacity_tb(6, 8, 8),
        "RAID10": raid_capacity_tb(1, 8, 8),
    }
    assert capacities == {"RAID0": 64, "RAID5": 56, "RAID6": 48, "RAID10": 32}

    path = IOPath(device_iops=480000, controller_iops=120000, link_iops=300000,
driver_iops=150000)
    assert path.bottleneck() == ("controller", 120000)

    transfer_bytes = 64 * 2**20
    dma_bandwidth = transfer_bytes / 0.020
    assert abs(dma_bandwidth - 3.3554432e9) < 1.0
    assert abs(500 / 3e9 - 1.6666666666666667e-7) < 1e-20

    print(f"average rotation latency: {rotation:.3f} ms")
    print(f"random sector read model: {access_ms:.3f} ms")
    print(f"availability: {avail:.9f}")
    print(f"expected downtime: {downtime_minutes:.3f} min/year")
    print("RAID capacity:", ", ".join(f"{name}={value} TB" for name, value in
capacities.items()))
    print(f"I/O bottleneck: {path.bottleneck()[0]} at {path.bottleneck()[1]:,} IOPS")
    print(f"DMA payload bandwidth: {dma_bandwidth / 1e9:.3f} GB/s")
    print("storage checks: PASS")


if __name__ == "__main__":
    main()
```

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study